

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	I want to do a GSoC project with Oppia because its mission of making high-quality education accessible to everyone aligns with my belief that technology should help reduce educational inequality. Contributing to Oppia allows me to work on features that can help make learning content available to people across different languages and regions. At the same time, Oppia's well-structured codebase and active open-source community provide a great opportunity for me to grow as a developer. Through this project, I hope to improve my skills in building scalable systems, collaborate with experienced contributors, and make meaningful contributions that support Oppia's goal of reaching more learners worldwide.	
What do you hope to learn/achieve during GSoC?	I aim to deepen my understanding of scalable backend architecture, specifically regarding caching mechanisms and database optimizations. Additionally, I want to refine my frontend skills in Angular by building intuitive UI components that seamlessly handle asynchronous API calls and complex validation states.	
Which Oppia teams have you collaborated with, and what have you done on those teams?		
	LEAP TEAM	1. Helped in triaging the issues of New Learners Dashboard. 2. Helped new Contributors to get started. 3. Helped Contributors with their coding related issues in the respective issue thread.
	CORE TEAM	1. Helped with the Co-Lead Works in the team. 2. Consistently kept the CORE LEAD dashboard count of unanswered issues below 20. 3. Helped in segregating issues for new contributors labeled good first issues.
Contact information	gamingzone.09990@gmail.com amiyapiyush3dec@gmail.com	
Preferred method of communication	Email	

Social media handles (optional)	https://www.linkedin.com/in/amiya-piyush-88794b288/
Which timezone will you primarily be in during the summer?	GMT +5:30
(If you are a student) When are your school holidays?	8 May - 2 August
What other obligations might you need to work around during the summer?	I won't be having any other obligations except work related to CORE TEAM
Planned time commitment	I plan to commit approximately 30hours per week during my holidays and 24 hours per week during school (with extension in hours if needed) to this medium-sized (~175 hours) project . I will balance my academic workload by strictly blocking out dedicated coding and communication windows for Oppia each day for 7 days a week.

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **May 25**. All work for Milestone 1 must be completed and submitted by **Jun 26** for internal feedback (with any revisions due by **Jul 10**), and all work for Milestone 2 must be completed and submitted by **Aug 19** for internal feedback (with any revisions due by **Sep 2**). We will not be able to extend these deadlines.

Coding period	I will adhere to the above deadlines.
----------------------	---

Communication Channels

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I can communicate through E-Mail and Google Chat

Contributor Briefing

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	• Yes
---	---

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	2.1. Automatic translation suggestions
Project size <i>(should match the options on the Project Ideas list)</i>	Medium (~175 hours)
Why did you choose this project?	I chose this project because it directly improves how fast and efficiently content can be translated on Oppia, which is important for making education accessible to more users. Language is a major barrier, and reducing manual effort in translation can significantly increase content reach. Technically, the project is interesting to me because it involves backend design , caching and integration with external services and building altogether a new Admin page. I love learning and this project feels like a crash course in software engineering. Hence it attracted me a lot. I have also been actively contributing to Oppia , so I am familiar with its codebase and workflows, which will help me execute this project effectively.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc.¹	https://docs.google.com/document/d/1XVS8t70iFSCmp_N_P-Hf5z_gDHTorrRU/edit?usp=sharing&ouid=107061560018785814530&rtpof=true&sd=true

I can debug and fix CI failures / flakes.² ³	1. https://github.com/oppia/oppia/pull/25536 2. https://github.com/oppia/oppia/pull/2487
WEB	
I can write Python code with unit tests.⁴	1. https://github.com/oppia/oppia/pull/24535 2. https://github.com/oppia/oppia/pull/25217 3. https://github.com/oppia/oppia/pull/24702
I can write TS + Angular code with unit tests.⁵	1. https://github.com/oppia/oppia/pull/24535 2. https://github.com/oppia/oppia/pull/24532 3. https://github.com/oppia/oppia/pull/23779
I can write or modify acceptance tests.⁶	1. https://github.com/oppia/oppia/pull/24702 2. https://github.com/oppia/oppia/pull/24702 (I first wrote E2E tests for this feature and they got LGTM. Later, based on reviewer feedback, I changed them into acceptance tests in the same PR, Hence mentioned it twice) 3. https://github.com/oppia/oppia/pull/25567
I can write or modify Beam jobs, with tests.⁷	1. https://github.com/oppia/oppia/pull/24825
I can figure out reproduction steps based on information given in the server logs.⁸	NA
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, and (c) play	NA

²

³

⁴

⁵

⁶

⁷

⁸

through the app locally. ⁹	
Write new Kotlin code with unit tests. ¹⁰	NA
Change Android UIs, write tests for them, and manually verify that they work. ¹¹	NA

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
NITISH	I worked together in the LEAP team where I learned about the contributor journey in Oppia, from first-time contributor to official member. He guided me on understanding the codebase and improving the quality of my pull requests.
MOSIN	Collaborated while working on Oppia issues and pull requests. Received guidance on fixing bugs, understanding review feedback, and following Oppia development practices.
HARDIK GOYAL	Worked together on debugging and resolving multiple issues in the Oppia repository. His feedback and suggestions helped me better understand Oppia's architecture and development workflow.

Section 3: Proposal Details

Problem Statement

Target Audience	1. Translation Contributors - Community members who translate lesson content into their native language using the Contributor Dashboard. 2. Translator Reviewers - Translators who approve the translation and after their review the translation becomes visible. 3. Oppia Administrators - Platform maintainers who manage translation configurations such as supported languages and translation providers.
-----------------	--

⁹

¹⁰

¹¹

Core User Need	The current translation process requires a lot of manual work and tasks to get the translations ready Hence we need a fast and smooth process to get the translations done without compromising the quality. Falling back to the API Providing AI Generated translation will reduce manual effort and allow contributors to focus on verifying and refining translations instead of writing them entirely from scratch..
-----------------------	--

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Admin Configuration and Feature Toggling	As a Translator Admin, I need to enable/disable the overall auto-translation feature, add/remove languages, and map them to specific providers, so that I have full control over the translation infrastructure and service costs.	1. Navigate to the Translator Admin page. 2. Enable or disable the automatic translation suggestions feature. 3. Add or remove supported languages for auto-translation. 4. Select third party service(e.g. Azure) for a specific language. 5. The configuration is saved automatically (Clicking "Add" immediately saves that specific row. Clicking "X" immediately deletes that specific row. Flipping the toggle immediately	https://drive.google.com/file/d/1mJUp6gfJEX-Vx2Md9LC4teiiRRotXpBh/view?usp=sharing

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
			updates the master feature flag.)	
2	Generate & Submit (Path A: Edited Text)	As a Contributor , I need to request an AI-generated translation suggestion, so that I can accept it (with or without edits) instead of translating manually from scratch.	1. Open a translation opportunity.	https://www.figma.com/design/15CN6GOxtqlZYKrOYBK88p/Automatic-translation-UI?node-id=0-1&p=f Flow 1
			2. Hover over "Auto-translate" to view the active state, then click. .	Flow 2 and 3
			3. View generated text and the source warning placed directly under the editor	Flow 4
			4. Edit the text if needed	Flow 6
			5. Click "Save and translate another".	
3	Generate and Submit (Path B: Unedited text)		1.Open translation opportunity	Flow 1
			2.Hover over "Auto-translate" to view the active state, then click.	Flow 2 and 3
			3.. View generated text and the source warning placed directly under the editor.	Flow 4
			4.. Click "Save and translate another".	
			5. A popup appears asking: "Are you sure this auto-generated	Flow 5

#	Title	User Story Description (role, goal, motivation) "As a ..., I need ..., so that"	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
			<i>translation needs no edits?"</i>	
			6.. Click "Yes, save" to confirm.	
4	Enforced Alt-Text Review (Image with Existing Alt-Text)	As a Contributor, I need to be forced to review translated alt-text for images that have existing alt descriptions before submitting, so that accurate image descriptions are maintained.	1. Generate an auto-translation containing an image that has existing alt-text.	Till flow 4
			2. Notice the "Save and translate" button is disabled and red text appears above it: <i>"Please verify that the translated alt text is accurate."</i>	Flow 4
			3. Click the image to open the alt-text modal.	
			4.. Review/edit and close the modal.	
			5. Submit the translation.	
5	Image without Alt-Text	As a Contributor, I need to be able to submit an auto-translated image without being forced to review alt-text if the original author did not require it, so that I can translate efficiently without unnecessary blockers.	1. Generate an auto-translation containing an image that has no alt-text. 2. Notice the "Save and translate another" button remains fully enabled and no red warning text appears. 3. Submit the translation directly (with or without editing the translation). 4. After this follow flow 5 or 6 as required.	https://github.com/oppia/oppia/discussions/25597

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
5	Preserving Math and LaTeX Content	As a Contributor, I need mathematical equations and proprietary Oppia math components to remain unchanged during auto-translation, so that mathematical expressions remain correct and readable.	1. Open a translation opportunity that contains an <oppia-noninteractiv e-math> component. 2. Click "Auto-translate". 3. The translation suggestion is generated. 4. Observe that the surrounding text is translated but the mathematical expression remains unchanged. 5. Review the translation. 6. Click "Save and translate another."	Mock link
6	Safe Translation of Hyperlinks	As a Contributor, I need hyperlink text to be translated while keeping the original URL unchanged, so that references and external resources remain valid.	1. Open a translation opportunity containing a hyperlink. 2. Click "Auto-translate". 3. The translation suggestion is generated. 4. Observe that the anchor text is translated but the hyperlink URL remains unchanged.	Mock link

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that"</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
			5. Inspect the hyperlink in the editor to confirm the original URL is preserved. 6. Click "Save and translate another."	
7	Manual Fallback / Opt-Out	As a contributor , I need to be able to ignore AI suggestions entirely, so that I can write a manual translation as I did before this feature existed.	1. Open a translation opportunity. 2. Ignore the "Auto translate" button 3. Type translation manually 4. Click "Save"	Current style in Oppia

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Translator Admin	Admin configures Azure for Hindi and enables the overall auto-translation feature.	Log in as a user with Translator Admin privileges. Navigate to the Translator Admin page.	1. Toggle the master "Enable Automatic Translations" switch to "ON". 2. Add 'Hindi' to the list of supported languages. 3. Select 'Azure' from the third-party provider dropdown for Hindi.	After step 4, the backend updates the automatic translation status and mapping in the singleton model, and the Auto-translate button appears for Hindi in the Contributor Dashboard. After step 5, the UI removes the row, the backend updates the mapping to remove Hindi, and the Auto-translate option stops coming for Hindi in the Contributors Dashboard.

			4. Click Add 5. Verify the row that appears in the table. 6. Click the "X" (Remove) button next to the newly added 'Hindi' row.	
Contributor	(Path A) Contributor generates an AI translation, edits it, and submits it smoothly.	Log in as a user with Contributor rights for Hindi. Navigate to the Contributor Dashboard and open a text-only translation opportunity.	1. Hover over the "Auto-translate" button to verify the active hover state, then click it. 2. Wait for the text editor to populate and verify the source warning banner appears below the editor. 3. Manually add a new sentence to the populated text. 4. Click the "Save and translate another" button.	The translation saves immediately . The UI successfully transitions to the next available translation card.
Contributor	(Path B) Contributor attempts to submit an unedited AI translation and must confirm via a popup modal	Log in as a user with Contributor rights. Navigate to the Contributor Dashboard and open a translation opportunity.	1. Click the "Auto-translate" button and wait for the text to populate. 2. Make absolutely zero edits to the text in the editor. 3. Click the "Save and translate another" button. 4. Verify the confirmation popup appears. 5. Click "Yes, save" inside the popup.	The system intercepts the initial save attempt and displays the modal asking : " Are you sure this auto-generated translation needs no edits?" The translation is only saved , and the UI only progresses to the next card, after the user explicitly clicks " Yes, save " in the modal
Contributor	Contributor is blocked from submitting an AI translation until the translated image alt-text is explicitly reviewed	Log in as a user with Contributor rights. Navigate to the Contributor Dashboard and open a translation opportunity that contains at least one image.	1. Click the "Auto-translate" button. 2. Attempt to click the "Save and translate another" button but fails to do so. 3. Click on the image in the editor	At step 2 , the save button is disabled and red helper text is displayed above it. At Step 5, the red helper text disappears, the save button becomes enabled , and the submission successfully saves.

			to open the alt-text modal. 4. Review the alt-text, then close the modal. 5. Click the "Save and translate another" button again.	
Contributor	Contributor generates an AI translation for a content block containing mathematical expressions.	Log in as a Contributor. Open a translation opportunity containing an <code><oppia-noninteractive-math></code> component.	1. Click Auto-translate. 2. Observe the populated translation in the editor. 3. Verify the mathematical expression renders correctly. 4. Click Save and translate another.	The mathematical expression remains unchanged and renders correctly after translation. The translation saves successfully.
Contributor	Contributor generates an AI translation for a text block containing a hyperlink.	Log in as a Contributor. Open a translation opportunity containing a hyperlink.	1. Click Auto-translate. 2. Inspect the translated content. 3. Verify that the anchor text is translated. 4. Verify that the hyperlink URL remains unchanged. 5. Click Save and translate another.	The anchor text is translated while the href URL remains identical to the source content.
Contributor	Contributor completely ignores the AI feature and translates manually (Opt-out fallback)	Log in as a user with Contributor rights. Navigate to the Contributor Dashboard and open a translation opportunity.	1. Ignore the "Auto-translate" button completely. 2. Manually type a translation into the rich text editor from scratch. 3. Click "Save and translate another".	The system behaves exactly same as before this AI translation was implemented.

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
1.	/generate-translation	POST	New	Takes {"source_text": ".....", "source_language_code": "en", target_language_code": "hi"} (PayLoad) Checks Datastore cache. If there is a cache miss, it uses the TranslationProviderRegistry to query the specific translation API configured for the target_language_code (such as . Azure or GCP) Translator API. Returns { "translated_text": "....."}
2	/translation-provider-mapping	GET,PUT	New	The GET request will return both the active mappings and the dictionary of human-readable names. It dynamically constructs this response by reading the active configuration from the Datastore and joining it with the static machine_translation_provider_display_names.json file. For example, it may return a response such as {"machine_translation_provider_mappings": {"hi": "azure", "es": "gcp"}, "available_providers": [{"id": "azure", "display_name": "Azure Translator" }, {"id": "gcp", "display_name": "Google Cloud Translate" }] }. The PUT request updates settings from the Translator Admin page. It reads the data and sends it to the service layer. If the request is about the master toggle (automatic_translation_is_enabled), it calls

				update_automatic_translation_status(). If the request is about language mappings (machine_translation_provider_mappings), it calls save_machine_translation_provider_mapping(). Both methods get the singleton model using MachineTranslationPolicyModel.get(MACHINE_TRANSLATION_POLICY_ID, strict=False). If it does not exist (for the first time), it creates it safely. Before saving with put(), the mappings are checked by creating a MachineTranslationProviderMapping object and calling validate() to make sure everything is correct.
3.	/suggestionhandler	POST	Existing	Updated to accept was_auto_generated, auto_generation_provider, and was_edited as optional keys inside change_cmd payload. Since the handler delegates validation to domain_objects_validator.validate_suggestion_change, no schema changes to the handler itself are needed; only the AddWrittenTranslationOnCmd domain object in exp_domain.py will be updated to accept these new optional fields.

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
1.	/generate-translation	POST	Called by the Contributor Dashboard frontend when a user click the " Auto translate " button to fetch the AI- generated translation dynamically
2.	/translation-provider-mapping	GET,PUT	Called by the Translator Admin frontend page to fetch the current configuration and save updates when the admin maps a language to a specific provider (e.g. Azure)

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.	AutoTranslationCacheModel (used for	A new dedicated AutoTranslationCacheModel (similar to MachineTranslationModel) will be defined	Existing translations in Oppia are unaffected.

	caching)	in core/storage/translation/gae_models.py as the caching layer for AI - generated translations. It stores source_language_code,target_language_code,source_text,translated_text.	
2	MachineTranslationPolicyModel(New)	A new singleton model will be created in core/storage/translation/gae_models.py, identified by the fixed constant MACHINE_TRANSLATION_POLICY_ID (set to 'machine_translation_policy'). This model stores the language_to_provider_mapping (as a JsonProperty) and the automatic_translation_is_enabled flag (as a BooleanProperty). It is accessed using the MachineTranslationPolicyModel.get(id, strict=False) method,similar to VoiceoverAutogenerationPolicyModel . Additionally, it follows standard BaseModel conventions by setting DELETION_POLICY = models.DeletionPolicy.NOT_APPLICABLE, since it stores system-level configuration rather than user-specific data."	N/A. New model , no migration needed

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	AdminProvider Mapping	Table and dropdowns in Translator Admin page to add/remove languages, select services , and toggle the master feature state	No(admin only)	Mock link	Will Ensure full keyboard navigability (using Tab/Enter keys) dropdowns, tables, and buttons.
2.	AutoTranslateButton	Button in Contributor Dashboard translation modal	Yes	Flow 1 2 3	Will Ensure standard

		with an explicit hover state			keyboard trigger support (Enter/Space)
3	AltTextVerificationPrompt	A message - Please verify that translated alt text is accurate	Yes	Flow 4	High color contrast for text , readable by screen readers
4	UneditedTranslationModal	Popup asking: "Are you sure this auto-generated translation needs no edits?" leading to the "Yes, save" button.	Yes	Flow 5	Focus trapped inside the modal when active; Escape key or a close button mapped to close it.

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	AutoTranslationCacheModel	source_language_code, target language ,, and the generated translated text	To reuse generated translations for identical content across explorations, reducing processing time, API calls, and external service costs	Yes. This data only contains public curriculum lessons content and language codes. It is completely decoupled from any user account or personal identifiable information.	N/A. This is a system level optimization for curriculum content , not user data	Not applicable. Since it does not contain user personal identifiable information and is not tied to a user ID, it is unaffected by user account wipeout requests.	Not exported. This is platform infrastructure/curriculum data, not user personal data.
2.	MachineTranslationPolicyModel	Admin-configured mapping of language codes to translation providers and master enable/disable flag	Required to persist admin configuration across server restarts.	Yes. Contains only language codes and provider names	N/A. System config not user data	Not_Applicable. Not tied to any user account	Not exported. Platform infrastructure data

Connection to Existing Work

Caching Layer: The project will introduce a new dedicated `AutoTranslationCacheModel` in `core/storage/translation/gae_models.py` to store and retrieve generated translations. Caching will happen after the post-processing and sanitization steps, meaning we store the final, ready-to-render HTML. This ensures cache hits require zero additional compute overhead.

Frontend Services: The feature will extend the existing `TranslateTextBackendApiService` ([core/templates/pages/contributor-dashboard-page/services/translate-text-backend-api.service.ts](#)) and the `TranslationModalComponent` ([link](#)) so that AI-generated suggestions fit naturally into the current contributor workflow.

Content Sanitization: After translation, the content will be cleaned using Oppia's existing `core.domain.html_cleaner.py` ([link](#)) method to ensure that no malformed HTML is stored.

Testing Approach: A `TranslateEmulator` will be created following the same pattern as the existing [cloud_translate_emulator](#) used in the codebase. This allows unit tests and CI to run without calling the real translation API.

Other Requirements

Testing and CI/CD

Unit Tests: Unit tests will be written for the HTML pre-processing and post-processing logic to check that `translate = "no"` is correctly added to Oppia components.

Acceptance Tests: End-to-end acceptance tests will be added to cover the main user flows, including edited translation submission, the confirmation popup for unedited translations, the alt-text review requirement, and the manual translation option.

API Emulation: A `TranslateEmulator` will be created to simulate responses from the Azure API during testing. This prevents external API calls during CI runs and keeps tests stable.

Alt-Text Review: The system will require contributors to review the translated alt-text for images before submitting a translation. This helps maintain accurate descriptions for screen-reader users.

UI Accessibility: New UI elements such as the "Auto-translate" button and the confirmation modal will include proper ARIA labels and support keyboard navigation.

Credential Management: Azure API keys will be stored using environment variables and accessed through [feconf.py](#) so that no sensitive information is hardcoded in the codebase.

Steps to get the API key

[Reference to official page](#)

Step 1: Create resource

Go to <https://portal.azure> -> Sign in and search "Translator" -> Click Create -> Fill details
Click Create and then Go to resource

Step 2: Get API key Open "Keys and Endpoint" -> Copy Key 1 and Region

Step 3: Set environment variables

AZURE_TRANSLATOR_API_KEY=<your_key>

Step 4: Use in code (feconf.py)
import os

AZURE_TRANSLATOR_API_KEY = os.environ.get('AZURE_TRANSLATOR_API_KEY')

AZURE_TRANSLATOR_REGION = "global" or whatever the region is

Section 3.2: HOW

Existing Status Quo

The Oppia platform currently relies on a manual,community-driven translation workflow to get the translations done.Contributors translate lesson content in the Contributor Dashboard and their translations are later reviewed and approved by translator admins. While this process helps maintain translation quality, it also slows down the process because translators need to write every translation from scratch.As a result, it takes longer for educational content to become available in multiple languages, especially for learners in regions where English is not the primary language.

Solution Overview

During my review of the Oppia codebase, I found that the platform already includes some basic support for machine translation. It has a caching model called MachineTranslationModel and a simple wrapper for Google Cloud Translation ([oppia/core/platform/translate/cloud_translate_services.py](https://github.com/oppia/oppia-core/blob/master/core/platform/translate/cloud_translate_services.py)). However, the current implementation is limited because it is hardcoded to use only Google Cloud, and supports only five languages with poor error handling.

To address this, this project proposes adding AI-generated translation suggestions to the Contributor Dashboard. The system will generate AI-based draft translation suggestions in the Contributor Dashboard to assist contributors during the translation process.Translators will still need to review , edit and approve the translations before submitting. This ensures that the existing human review process remains unchanged.

[On the backend, I will implement a TranslationProviderRegistry in core/domain/machine_translation_services.py.](#)

To make adding and removing translation service providers safe and scalable, I will implement a Strategy pattern using Python's Abstract Base Classes module. I will define a new abstract base class, `BaseTranslationService`, inside `core/platform/translate/base_translate_services.py` (New). This class will act as a strict contract that all translation providers must follow, defining abstract methods (like `generate_translation`) that any new provider must implement.

The `TranslationProviderRegistry` will type-check and interact only with this `BaseTranslationService` interface. Since App Engine environments are stateless, the provider configuration will follow a dual-layer approach.

First, to manage the provider configurations and map internal identifiers to human-readable UI text, I will maintain two static JSON files :

`machine_translation_providers.json` (similar to [autogeneratable_language_accent_list.json](#)) : This acts as a whitelist defining which translation providers technically support which language (e.g., `{"hi": ["azure", "gcp"]}`).

`machine_translation_provider_display_names.json` (similar to [language_accent_master_list.json](#)) : It acts as a static dictionary mapping internal machine IDs to human-readable names (e.g., `{"azure": "Azure Translator"}`). By keeping display names entirely out of the Datastore, we prevent UI-level typos and avoid complex database migrations if a third-party service ever rebrands. The GET `/translation-provider-mapping` endpoint reads this file to supply the frontend with the `display_name` options for the dropdown menus.

The active admin configuration will be stored in a new singleton datastore model called `MachineTranslationPolicyModel`, directly following the `VoiceoverAutogenerationPolicyModel` pattern in `core/storage/voiceover/gae_models.py`. This model uses a fixed constant ID, stores `language_to_provider_mapping` as a `JsonProperty` dict, and `automatic_translation_is_enabled` as a `BooleanProperty`.

The singleton model is accessed using `MachineTranslationPolicyModel.get(MACHINE_TRANSLATION_POLICY_ID, strict=False)`, following the same pattern as `VoiceoverAutogenerationPolicyModel`. When `strict=False` is used, the method returns `None` instead of raising an exception if no instance exists.

The service layer is responsible for handling creation and persistence of the singleton. To support independent, immediate saves from the admin UI, datastore updates are split into two methods: `update_automatic_translation_status` (for the master toggle) and `save_machine_translation_provider_mapping` (for the dictionary mapping). Following Oppia's standard patterns, both methods gracefully handle first-time initialization by instantiating a new model with the fixed ID if `MachineTranslationPolicyModel.get(..., strict=False)` returns `None`.

The `TranslationProviderRegistry` reads this configuration at runtime by calling `machine_translation_services.get_machine_translation_provider_mapping()`.

Oppia lessons often have special content like math expressions and custom HTML tags. To make sure translation services do not change them, the backend will add the `translate="no"` attribute to these tags before sending the text to Azure.

After receiving the translated text, a post-processing step will run to strip the temporary `translate="no"` attributes. Oppia's robust Angular frontend already handles Right-to-Left (RTL) text directionality dynamically via CSS and UI configuration.

To improve performance and reduce repeated calls to the translation API, generated translations will also be cached in the datastore. A cache model will store translations using `utils.convert_to_hash` (as Oppia already uses it) to generate a deterministic lookup key based on the source language code, the original text, and the target language code.. If the same content needs to be translated again, the cached version can be returned immediately instead of calling the external API again. After retrieving a cached entry, the system compares the stored `source_text` with the incoming request to avoid hash collision issues. To securely authenticate with the Azure Translator API, the API keys will be stored as environment variables and accessed through [feconf.py](#). This ensures that sensitive credentials are not hardcoded in the repository.

Flow: Frontend Request -> Handler -> Cache Check (Hit/Miss) -> Pre-processing (BeautifulSoup `translate="no"`) -> Azure API -> remove added attribute (`translate = "no"`) using regex -> `html_cleaner` sanitizes the result -> Store in Cache -> Return to Frontend

Third-Party Libraries

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)
1.	Azure-ai-translation-text (latest) Proof that it is Compatible with Oppia	Azure	used to connect with the Azure Translator API to generate AI translation suggestions..	MIT
2.	BeautifulSoup 4 (Existing in Oppia)	BeautifulSoup4	The pre-processing step uses BeautifulSoup4 to scan the HTML and identify all tags whose names begin with <code>oppia-noninteractive-</code> . For each of these tags, it adds the attribute <code>translate="no"</code> so that translation systems ignore their content. Since this approach is based on a prefix pattern rather than a fixed list, any new components introduced in the future that follow the same naming convention will automatically be excluded from translation, without requiring any updates to the translation	MIT

			service.	
--	--	--	----------	--

...

“Service” Dependencies

No.	Dependency name	Why it is needed	What our plan is, if the dependency fails under us
1	Azure Translator API	Used to generate AI-based translation suggestions for lesson content.	Error handling will follow the same pattern as in voiceover_regeneration_services.py. The API call is wrapped in a try/except block. If any error happens (like the service being unavailable), it is caught and stored in error_details. If error_details is not None, an exception is raised and sent to the handler. The handler then sends an error response to the frontend, which shows a notification and lets the contributor continue with manual translation.

Key High-Level and Architectural Decisions

Decision 1: Preventing Data Corruption in HTML, Mathematical Content and similar

I have considered the following alternatives to safely translate that contains rich text:

1. **Alternative 1(chosen) Azure based** - Send the text to the Azure API using textType - html and mark protected elements such as <oppia-noninteractive-math> with HTML attributes like [translate = “no” or class = “notranslate”](#) so the translation engine skips them.

After getting the translated text, we simply strip the temporary translate=“no” shield. We rely on Oppia's existing Angular frontend infrastructure (which dynamically applies .rtl.css and sets languageDirection in the UI config) to handle Right-to-Left rendering natively. This maintains a strict separation of concerns between backend data and frontend presentation.

2. **Alternative 2 Placeholder based**- Use an HTML parser to extract sensitive elements (e.g., <oppia-noninteractive-math>), replace them with temporary placeholder tokens before sending the text to the translation API, and restore the original elements after the translated response is received.

Among these, I believe that Alternative 1 is the best approach, because:

- Leverages native API features: Azure supports the translate = “no” attribute for HTML inputs, so using it reduces the need for complex backend parsing.
- Avoids token corruption: Using placeholder tokens (Alternative 2) can be risky because the translation system might modify or misplace the tokens, breaking reconstruction. Alternative 1 avoids this issue completely.

The above approaches are contrasted in detail in the following table:

	Alternative 1: Azure based	Alternative 2: Placeholder based
Approach	Use translate = "no" where needed, remove it after translating, and let Oppia's Angular frontend handle right-to-left (RTL) display.	Extract sensitive elements , replace with placeholder, and restore after translation
LaTeX/Math Safety	Safe	Safe
Handling Oppia components	translation engine respecting attributes	Fully controlled by backend preprocessing
Implementation complexity	Low (simple integration + light post-processing)	Moderate(requires HTML parsing and placeholder mapping)
Reliability	High in practice	platform-controlled
Context of semantic tags	Preserved and visible to translation api	Lost as translation api sees placeholders
Risk of Token issues	None	Possible if placeholder are modified or misplaced by the translation system

Decision 2: Reusing MachineTranslationModel vs Creating a New Cache Model

To avoid repeated API calls and improve performance, translations will be cached in the datastore. The alternative here is whether to reuse Oppia's existing MachineTranslationModel as the cache store, or introduce a dedicated new model. I considered the following alternatives:

I considered the following alternatives for defining the primary key used to query the MachineTranslationModel :

1. Create a new AutoTranslationCacheModel in core/storage/translation/gae_models.py to store and retrieve generated translations.
2. Reuse Oppia's existing [MachineTranslationModel](#) present in core/storage/translation/gae_models.py.

Among these, I believe that **Alternative 1 Introducing New Cache Model** is the best approach, because:

- Creating a new Cache model gives a full liberty on the project and keeps it separated. Moreover, android_services.py unconditionally wipes ALL entries on topic deletion. ([proof](#)). This makes it fundamentally unsafe as a persistent cache - any topic deletion silently destroys all cached translations.
- Alternative 2 is also not a right move because the model may contain legacy GCP - generated entries that could be silently served as Cache hits for new translations.

The above approaches are contrasted in detail in the following table:

	Alternative 1: Introducing New Cache Model	Alternative 2: Reusing MachineTranslationModel
Datastore Schema	Introduces a new datastore model for caching translations	Reuses the existing datastore model already implemented in Oppia
Development Effort	Moderate implementation effort due to defining a new model with clean schema	Lower implementation effort since the existing model already supports deterministic caching
Codebase Consistency	Creates duplicate infrastructure for machine translation caching	Aligns with existing Oppia translation infrastructure
Legacy GCP pollution	No existing entries	Might be not compatible with existing cached translations, as old cache may persist
Implementation Complexity	Moderate	Low
Android Wipe out Risk	Completely isolated from it	Cache will be wiped on every topic deletion
Beam Jobs	Not need in this project	Deletion and Migration jobs will be needed

Decision 3: Using Platform Parameters vs Creating a Dedicated Datastore Model for Provider Mapping

Alternative 1(Chosen): Dedicated Datastore Model

To store the mapping between languages and their translation providers, I considered creating a dedicated datastore model called MachineTranslationPolicyModel similar to VoiceoverAutogeneratedPolicyModel. This model would store a dictionary mapping languages to their corresponding translation providers along with any related configuration metadata. The model would be defined in core/storage/translation/gae_models.py

Alternative 2: Platform Parameters

This involves registering a new parameter named MACHINE_TRANSLATION_PROVIDER_MAPPINGS in core/domain/platform_parameter_list.py and defining it in core/domain/platform_parameter_registry.py. The parameter will store a dictionary mapping language codes to their respective translation providers (for example, { "hi": "azure", "es": "azure" }).

Using the Platform Parameter system allows the configuration to be managed through Oppia's existing infrastructure without introducing a new datastore model or additional audit Beam jobs. The TranslationProviderRegistry will read this parameter to determine which translation provider should be used for a given target language.

Out of these two Alternative 1 is better as (comparison made in table below)

The above approaches are contrasted in detail in the following table:

	Alternative 1: Dedicated Datastore Model	Alternative 2: Platform Parameters
Pattern	Follows VoiceoverAutogenerationPolicyModel	No existing dict param to follow
Primary validation	Domain object validate() in translation_domain.py	None
Secondary validation	Domain-level validate() rejects invalid configs at write time	Not applicable
Codebase Consistency	High	Low - introduces unverified pattern
Beam jobs required	Audit jobs will follow post-GSoC; the validate() call at write time is sufficient protection during this project's scope.	Not required

Decision 4: Standardizing Provider Interfaces via Abstract Base Classes

To ensure the backend is not tightly coupled to Azure and to make adding future translation providers (like GCP or AWS) safe and seamless, I considered how to structure the platform-layer adapters.

Alternative 1 (Chosen): Abstract Base Classes (Strategy Pattern)

Introducing an abstract base class BaseTranslationService using Python's abc module. All new translation providers must inherit from this and implement strict @abc.abstractmethod functions.

Alternative 2: Ad-Hoc Implementations

Allow each provider file to define its own methods and handle the differences inside the domain-layer registry.

Among these, Alternative 1 is the best approach because it heavily enforces the Open-Closed Principle. The TranslationProviderRegistry will only interact with the BaseTranslationService interface. The core

machine_translation_services.py file will never need to know the specific implementation details of Azure(or any other provider) . By using an Abstract Base Class (BaseTranslateServices), Python forces every provider to implement the required method (like generate_translation). If they don't, Python throws an error immediately – so correctness is enforced by the language, not just by convention.

	Alternative 1: Abstract Base Classes	Alternative 2: Ad-Hc implementation
Design Pattern	Structured (Strategy Pattern, OOP)	Unstructured (Procedural)
Adding New Provider	Easy and safe (must follow defined methods)	Risky (no strict rules, inconsistent code)
Domain Layer impact	Clan (no if/else need)	Messy
Codebase Consistency	High (standardized approach)	Low (difficult styles , technical debt)
Scalability	High (easy to extend)	Low (hard to maintain as system grows)

Risks and mitigations

Potential Risk	Mitigation
Accessibility degradation : Contributors may blindly accept AI-generated translations without verifying the alt-text of images, which could lead to incorrect descriptions for visually impaired learners.	The Angular frontend (oppia/core/templates/pages/contributor-dash-board-page/modal-templates/translation-modal.component.ts) will parse the translated HTML and detect image components. A validation mechanism will require contributors to explicitly review image alt-text before submission. The “Save” button will remain disabled until each image alt-text is reviewed through the alt-text modal.
HTML or LaTeX may get corrupted during translation, or equations may appear reversed in RTL languages	The backend will use Azure's translate = “no” attribute to skip Oppia text rich components (e.g. <oppia-noninteractive-math> , <oppia-noninteractive-image> <oppia-noninteractive-video> <oppia-noninteractive-link> <oppia-noninteractive-skillreview>).The translated text will be stored as it is in the database. The pre-processing step uses BeautifulSoup4 to scan the HTML and identify all tags whose names begin with oppia-noninteractive-. For each of these

	tags, it adds the attribute <code>translate="no"</code> so that translation systems ignore their content. Since this approach is based on a prefix pattern rather than a fixed list, any new components introduced in the future that follow the same naming convention will automatically be excluded from translation, without requiring any updates to the translation service. To prevent RTL rendering corruption, the backend delegates directionality entirely to Oppia's existing Angular frontend infrastructure (which dynamically applies <code>.rtl.css</code> and sets <code>languageDirection</code> in the UI config), maintaining strict separation of concerns.
CI tests or local development may accidentally call the real Azure Translator API, which could slow down tests, require API credentials, or incur external service costs.	Unlike <code>CloudTranslateEmulator</code> which is a standalone plain class, <code>AzureTranslateEmulator</code> will inherit from <code>BaseTranslateServices</code>, ensuring the emulator is always guaranteed to match the same interface as the production implementation." It will not perform real translations but instead return predictable dummy data . It works in three ways: 1.It contains a pre-generated dictionary with hardcoded responses for common test inputs. 2.If an input is not found in the dictionary, it returns a default static response. 3.It provides a method like <code>add_expected_response()</code> that allows tests to inject custom mock responses, which is useful for testing pre-processing and post-processing logic. Local dev server - In models.py the registry will be extended by adding an <code>import_machine_translate_services()</code> . This method checks if the system is running in emulator mode and returns either the development or production implementation.
Hash collisions in the caching layer: Although very rare - different strings could theoretically generate the same hash, causing incorrect cache retrieval	After retrieving a cached entry, the backend will perform a secondary equality check on the stored <code>source_text</code>. If the text does not match, the system will treat it as a cache miss and generate a fresh translation.
Datastore growth from cached translations: Over time , caching translations for many strings may increase datastore storage usage.	Periodic cleanup policies can be introduced if necessary(a simple cron job that clears cache entries not accessed in the last 12 months by

	storing in AutoTranslationCacheModel, may be done in future work) .
Singleton entity missing on first deployment	is_automatic_translation_enabled() returns False and get_machine_translation_provider_mapping() returns an empty dictionary if no data is saved yet, following a safe default approach similar to voiceover services. Instead of crashing, the handler raises an InvalidInputException.
External service outage: if the Azure translation service becomes unavailable or times out, contributors may not receive translation suggestions.	The frontend will display a clear error notification and allow contributors to continue using the existing manual translation workflow without disruption
Deep Dive: Handling Third-Party Translation Service Failures https://learn.microsoft.com/en-us/azure/ai-services/translator/service-limits	Because this feature operates in real-time while a user is waiting at the UI , the error handling strategy prioritizes failing fast , degrading gracefully , and never blocking the core manual translation workflow
1.Missing or Invalid API Key (Auth Errors - HTTP 401) In ai_translate_services.py , the system checks that feconf.AZURE_TRANSLATOR_API_KEY exists before making the HTTP call. If it is missing, it raises an exception immediately. If the call is made and Azure returns a 401, the error message is extracted and returned up the chain.	Keys are securely stored as environment variables accessed through feconf.py and validated at startup to prevent runtime auth crashes
2.Rate Limiting (HTTP 429) & Quota Exhaustion (HTTP 403) The platform layer checks the HTTP response status. On 429 or 403, it sets a specific error_details string (e.g., 'Azure rate limit exceeded'). The domain service catches this, raises an exception, and the handler returns a structured error to the frontend. The UI displays the notification, and the user seamlessly falls back to manual translation.	The AutoTranslationCacheModel acts as the primary shield, ensuring identical text is never sent to Azure twice, drastically reducing API call volume. Additionally, admins can toggle the feature off via MachineTranslationPolicyModel if quotas are unexpectedly exhausted. An email will be sent to translation Tech Leads regarding this failure using the same email sending mechanism as automatic voiceover failure do. An example of a similar thing is this .

3.Service Outage / Unavailability (HTTP 503) A 503 is caught by a broad except Exception block in the domain service. The error is captured, raised, and pushed to the frontend as a structured error response.	The UI degrades gracefully. Contributors are notified of the outage but can continue translating manually without being blocked.
4.Network Timeout A requests.Timeout exception is caught by the same domain-level exception block.	The backend will enforce a strict timeout (10 seconds) on the HTTP request. This ensures a contributor's frontend session does not hang indefinitely waiting for Azure to respond.
5.Text Too Long / Bad Request (HTTP 400) If the content exceeds Provider's (e.g. Azure) character limits, it returns a 400. The platform layer intercepts this and surfaces a descriptive error to the frontend.	Azure returns 400050 or 400077 with a descriptive message. The platform layer reads the error code from the JSON response body, sets error_details, and propagates it up the chain.

Key Objectives for Each Milestone

Milestone	Key Objective
Milestone 1	The backend infrastructure for automatic translation suggestions is fully implemented . The system can generate translations using the Azure Translator API and store them in the datastore cache using deterministic hashing . The Translator Admin page allows administrators to enable or disable the feature , manage supported languages, and map languages to translation providers. Furthermore, the entire architecture is designed using abstract base classes to ensure that adding new translation providers in the future is seamless.
Milestone 2	Contributors can generate AI-based translation suggestions directly from the Contributor Dashboard . The interface allows users to review , edit , or reject the generated translations. Accessibility safeguards ensure that translated image alt-text is reviewed before submission

Implementation Approach

Storage Model Layer Changes

A new dedicated AutoTranslationCacheModel will be defined in core/storage/translation/gae_models.py as the caching layer for AI-generated translations. The system will first attempt to retrieve a cached translation using the deterministic datastore ID generated from the source text, language codes. If a cached translation exists, it will be returned immediately. Otherwise, the backend will call the configured translation provider and store the generated translation in the datastore for future reuse.

A new feature flag ENABLE_AUTOMATIC_TRANSLATION_SUGGESTIONS will be added to core/feature_flag_list.py. This acts as a developer-controlled switch during development and roll out.

The automatic_translation_is_enabled field in MachineTranslationPolicyModel is an admin-controlled switch that can be managed from the Translator Admin UI.

Storage Model Migrations

No datastore migrations are required. AutoTranslationCacheModel is a brand new model with no existing entries. MachineTranslationModel is left completely untouched. Since the model stores only curriculum content and is not tied to user accounts, it correctly uses the DELETION_POLICY.NOT_APPLICABLE policy.

Domain Objects

TranslationProviderRegistry

Implemented in core/domain/machine_translation_services.py (new file). This registry uses a dual-layer configuration. First, it reads a static whitelist (assets/machine_translation_providers.json) to determine which APIs (for example Azure or GCP) technically support which ISO-639-1 language codes. Second, at runtime it fetches the active, admin-selected mappings by calling get_machine_translation_provider_mapping(), which internally calls MachineTranslationPolicyModel.get(MACHINE_TRANSLATION_POLICY_ID, strict=False) and reads the language_to_provider_mapping field. The registry calls get_machine_translation_provider_mapping(), which returns an empty dictionary if no configuration exists.

The asset/machine_translation_provider_display_names.json file serves as the single source of truth for human-readable provider names. The backend handler for GET /translation-provider-mapping reads this file at request time and includes the full name mappings in the available_providers field of the response. The TranslatorAdminBackendApiService fetches this on page load and passes the list to the admin page component, which binds it directly to the provider selection dropdown. This means adding a new provider only requires updating the JSON file, no frontend code changes are needed.

The registry then returns a safe default (None or empty mapping), and the handler decides how to handle this case.

Based on the target language code, the registry selects the appropriate provider implementation. This architecture separates developer-maintained API limits from dynamically editable admin preferences, allowing new providers to be integrated without modifying the core translation logic.

To onboard a new translation provider in the future, a developer only needs to:

1. Add the new provider's supported languages to `machine_translation_providers.json` and id to name matching in `machine_translation_provider_display_names.json`.
2. `core/platform/translate/` that inherits from `BaseTranslationService` and strictly implements its abstract methods.
3. Register it in the `TranslationProviderRegistry`
4. Add its API credentials to `feconf.py` as environment variables
5. Extend the `TranslateEmulator` to support the new provider for CI/CD testing
6. Write unit tests for the new adapter

Because the Registry only communicates with the Abstract Base Class interface, no changes to the handler, domain service, caching layer, admin UI, or frontend are needed. The admin can immediately select the new provider from the dropdown for any supported language.

Service Method

`save_machine_translation_provider_mapping()`

To support quick updates from the admin UI, the datastore is updated using two separate methods. Both methods use `MachineTranslationPolicyModel.get(MACHINE_TRANSLATION_POLICY_ID, strict=False)` to get the model (same as we do in [voiceover_services.py](#))

If it does not exist, they create it safely.

1. `update_automatic_translation_status(automatic_translation_is_enabled: bool)`

This method handles turning the feature on or off. It gets or creates the model, updates the boolean flag, updates the timestamps (calling `update_timestamps()`), and saves it using `put()`.

2. `save_machine_translation_provider_mapping(language_to_provider_mapping: Dict[str, str])`

This method handles adding or removing language-provider mappings. It gets or creates the model, converts the input into a `MachineTranslationProviderMapping` domain object, and calls `validate()` to make sure the data is correct (valid language codes, valid providers, and correct provider-language mapping). After validation, it updates the mapping, updates timestamps (calling `update_timestamps()`) and saves it using `put()`.

MachineTranslationProviderMapping (Domain Object)

`MachineTranslationProviderMapping` is a simple domain class added in `core/domain/translation_domain.py`. It takes only a mapping of language codes to provider identifiers (as a dictionary). It has a `validate()` method that checks whether the input is a dictionary, whether each language code is valid using `utils.is_valid_language_code()`, and whether each provider supports the corresponding language by checking against `machine_translation_providers.json`.

The service layer always calls `validate()` before saving, ensuring invalid configurations are rejected at write time making a dedicated audit Beam job unnecessary for this project's scope. A periodic audit job may be written after project completion..

Translation Suggestion Metadata

The translation suggestion will include extra information that shows whether the translation was generated automatically and which service generated it. This will be seen by reviewers who review the translation for extra safety and future analytics of this feature. Instead of changing the existing SuggestionModel database schema, this information will be stored inside the change_cmd dictionary of the suggestion payload. This helps avoid complex database migrations.

The [change_cmd](#) sent by the frontend [TranslateTextBackendApiService](#) (oppia/core/templates/pages/contributor-dashboard-page/services/[translate-text-backend-api.service.ts](#))

The change_cmd payload will be extended to include additional metadata fields: was_auto_generated, auto_generation_provider, and was_edited, allowing the system to track whether a translation was generated by AI, which provider was used, and whether the user modified it. To support these fields without causing backend validation errors, updates will be made in core/domain/exp_domain.py by adding them to the optional_attribute_names list for CMD_ADD_WRITTEN_TRANSLATION in the ALLOWED_COMMANDS configuration. The AddWrittenTranslationCmd class will also be extended to define these fields as optional attributes with appropriate types (boolean for was_auto_generated and was_edited, and string for auto_generation_provider). In addition, corresponding updates will be made to the frontend TypeScript interfaces so that Angular can recognize, store, and correctly pass these new properties in the request payload to the backend.

User Flows (Controllers and Services)

MachineTranslationGenerateHandler (POST)

It handles the request when a contributor clicks the Auto Translate button on the Contributor Dashboard.

The frontend sends a request like this:

```
{
  "source_text": "...",
  "source_language_code": "en",
  "target_language_code": "hi"
}
```

Before starting the translation, the handler checks two conditions.

First, it checks whether the ENABLE_AUTOMATIC_TRANSLATION_SUGGESTIONS feature flag is enabled. This ensures that the feature has been properly released in the system.

Second, it calls machine_translation_services.is_automatic_translation_enabled service method to check if the admin has enabled the feature. If the singleton entity does not exist (admin has never configured the feature), this method returns False by default.

If both checks pass, the handler validates the request and sends the task to a domain service located in core/domain/machine_translation_services.py.

ERROR HANDLING FLOW

First, in `ai_translate_services.py`, the system calls **for example Azure** with a timeout. If Azure gives an error (like rate limit, quota issue, or server problem), this layer does not crash. Instead, it returns two things: no result and an error object containing the error code and message.

Then in `machine_translation_services.py`, this error is received. If the error is serious (like too many requests or quota finished), it sends an email to the admins so they know something is wrong. For other errors, it simply raises an exception so the next layer can handle it.

Next, in `MachineTranslationGenerateHandler`, the exception is caught. Instead of letting the backend crash (which would give a 500 error), it sends a proper JSON response with the error message. So the server stays stable.

After that, in `translate-text-backend-api.service.ts`, the frontend receives this response. If it sees an error, it treats it as a failure. It returns a clean result saying the translation failed.

Finally, in [translation-modal.component.ts](#) the error is made to show on screen.

Domain Service Orchestration (generate_and_cache_translation)

The domain service then performs the main work. It first checks if the translation already exists in the cache. Then it prepares the text for translation by injecting the `translate = "no"` attribute into protected components. After that, it sends the text to the configured translation provider, such as Azure, to generate the translation. Upon receiving the translated response from the provider, the service runs a Regular Expression to strip the temporary `translate="no"` attributes using `Regex`. The stripped text is then passed through `html_cleaner.clean()` to sanitize the output before it is stored in the cache and returned to the frontend.

Finally, the handler returns the translated text

The `translation_provider` field is returned so that the Angular frontend can store this information and include it in the `change_cmd` payload when the contributor submits the translation for review.

Pre-processing (Component Protection)

Pre-processing (Component Protection) & Input-Output Contract

Before sending the content to the translation API, the backend must ensure that Oppia's custom components are not modified or corrupted by the translation engine. To ensure high testability, this is implemented as a pure, idempotent function.

The Input-Output Contract:

Input (source_html): A string containing the raw HTML/text from the exploration state.

Output (protected_html): A string where all proprietary Oppia tags have been injected with the HTML5 `translate="no"` attribute.

Eg -

<p>Find the area: <oppia-noninteractive-math math_content="πr²"></oppia-noninteractive-math></p>

**<p>Find the area: <oppia-noninteractive-math math_content="πr²"
translate="no"></oppia-noninteractive-math></p>**

Execution Steps:

DOM Parsing: The source_html string is parsed using `bs4.BeautifulSoup(source_html, 'html.parser')`.

Dynamic Tag Discovery: The parser scans the tree using a regular expression prefix matcher: `soup.find_all(re.compile(r"^oppia-noninteractive-"))`. This guarantees that future components are automatically protected without maintaining a hardcoded list.

Attribute Injection: The code goes through all matched tags. To make sure it is idempotent, it first checks whether the tag already has `translate="no"`. If not, it adds it. This way, if the function runs again, it won't add the same attribute twice.

Hyperlinks: `<a>` tags are intentionally ignored, as the Azure API (using `textType="html"`) natively translates anchor text while preserving the href URL byte-for-byte.

Serialization: The modified DOM tree is serialized back into a string and returned.

This is done strictly using `soup.decode_contents()` rather than `str(soup)`. Because Oppia's content consists of HTML fragments rather than full documents, using `str(soup)` is risky as BeautifulSoup may attempt to auto-inject standard `<html>` and `<body>` wrappers. `decode_contents()` safely outputs only the inner HTML fragments exactly as they are needed, ensuring the frontend rendering is never broken.

BeautifulSoup's `find_all` method searches the entire HTML tree, so even if Oppia components are deeply nested inside other elements, they are still detected and protected. This ensures the structure remains correct, unlike simple regex-based approaches which may fail in such cases.

Cache Lookup

The system generates a deterministic datastore key for `AutoTranslationCacheModel`:
`[source_language_code].[target_language_code].[utils.convert_to_hash(source_text, 50)]`

(Note :The provider is intentionally excluded from the cache key. Translations are cached based only on the source text and language pair, allowing results generated by one provider to be reused by others. This improves cache hit rate, reduces redundant API calls, and keeps the caching layer provider-agnostic. But, if provider-specific separation is required, the provider name can be easily incorporated into the cache key to isolate entries.)

The system first checks whether a translation already exists in the datastore.

If a cached translation is found and the stored `source_text` matches the incoming request, the cached result is returned immediately. This avoids repeated calls to the external translation API for identical text.

Translation Provider Selection

If no cached translation exists, the backend uses a `TranslationProviderRegistry` in `core/domain/machine_translation_services.py`. This registry fetches the active configuration using `get_machine_translation_provider_mapping()`, which internally calls `MachineTranslationPolicyModel.get(MACHINE_TRANSLATION_POLICY_ID, strict=False)`.

The retrieved mapping (e.g., `{"hi": "azure", "es": "azure"}`) is validated against a static whitelist (`machine_translation_providers.json`) to ensure the selected provider supports the target language.

Based on this, the registry selects the appropriate provider. This design enables dynamic provider switching without server restarts and allows easy addition of new providers without modifying core logic.

API Translation

Following Oppia's architecture, the platform layer handles external communications. To ensure scalability, I will first introduce a `BaseTranslationService` abstract class in `core/platform/translate/base_translate_services.py`. This class will use the `@abc.abstractmethod` decorator to define a strict contract (e.g., a `generate_translation` method) that all translation providers must follow.

The Azure Translator API integration will then be implemented in `core/platform/translate/azure_translate_services.py` (or generic file naming `core/platform/translate/ai_translate_services.py`) - . This specific Azure adapter class will inherit from `BaseTranslationService`. When a translation is not found in the cache, the domain layer will call this platform service to get the translation. Because the domain layer only interacts with the abstract interface, the core logic remains completely decoupled from Azure's specific API design.

Credentials will be securely managed using environment variables and accessed through [feconf.py](#) ensuring that no sensitive API keys are hardcoded in the repository.

Post-processing (Attribute Stripping and Sanitization)

Once the translated text is returned by the translation provider, it must be cleaned and formatted before being stored in the cache or returned to the user.

1. Attribute Stripping: The `translate="no"` attributes that were injected solely for Azure's benefit during pre-processing are stripped out using Regexp so they do not pollute Oppia's datastore schema.

Since pre-processing already uses `decode_contents()` to produce a clean fragment, the post-processing step only needs a lightweight Regexp pass to remove `translate="no"` , there is no need to re-parse the Azure output with BeautifulSoup.

2. HTML Sanitization: Even though the external API returns valid HTML, the string is passed through Oppia's standard `core.domain.html_cleaner.clean(translated_text)` to strip any disallowed tags, normalize whitespace, and guarantee the payload is 100% safe to render on the frontend. The final sanitized string is then stored in the `AutoTranslationCacheModel`.

[`core.domain.html\_cleaner.clean\(\)`](#)

Cache Storage

The fully sanitized translated result is stored in `AutoTranslationCacheModel` using the deterministic datastore ID generated earlier.

This allows future requests for identical content to be served directly from the datastore cache instead of calling the external translation API again, significantly reducing latency and API usage.

Response to Frontend

Finally, the handler returns the translated content to the frontend

The Contributor Dashboard then populates the translation editor with the generated suggestion for review and editing.

Web frontend changes

Frontend Architecture for Translator Admin Page

To implement the administrator configuration interface in a maintainable way, the feature will follow Oppia's Angular modular architecture pattern. A new Angular module will encapsulate all components and services related to translation configuration.

`oppia/core/templates/pages/Translate-admin-page/`

TranslatorAdminPageModule

A new Angular module will be created to organize the translation administration functionality. This module will declare all components related to managing translation providers and supported languages.

The module will include:

1. TranslatorAdminAuthGuard

- Responsible for restricting access to the Translator Admin page.

- Ensures that only users with translator admin privileges can access configuration settings.
- If an unauthorized user attempts access, they will be redirected to an appropriate page.

2. TranslatorAdminPageComponent

This will act as the main “smart component” for the page.

Responsibilities include:

- Fetching the current language-to-provider configuration from the backend Displaying the list of supported languages in a table.
- Opening the provider selection modal when an admin edits a language configuration Sending updated configuration to the backend.
- Dynamic UI Data Binding: The human-readable provider names originate from the static `machine_translation_provider_display_names.json` file on disk, which acts as the single source of truth. The GET `/translation-provider-mapping` handler reads this file at request time and includes the full name mappings in the `available_providers` field of the response — for example `{id: "azure", display_name: "Azure Translator"}`

The component maintains strict separation between logic and presentation. When the page initializes, it receives the `available_providers` list from the backend and transforms it into a key-value lookup map (e.g., `Record<string, string>`) in memory. When rendering the table, it uses this map to instantly resolve the active machine id to its corresponding `display_name` (e.g., mapping `"azure"` -> `"Azure Translator"`) in $O(1)$ time, dynamically outputting the full name into the HTML template without ever storing it in the database. Following the same pattern as `VoiceoverAdminPage`, the GET call to `/translation-provider-mapping` is made fresh on every `ngOnInit()` of `TranslatorAdminPageComponent` — there is no module-level or global cache. This ensures the dropdown always reflects the latest backend state when the admin loads the page.

3. EditTranslationProviderMappingModalComponent

- A reusable modal component responsible only for editing provider mappings.
- Displays a dropdown allowing the admin to choose a translation provider (e.g., Azure).
- Returns the selected provider to the parent component when saved.

4. TranslatorAdminBackendApiService

- A dedicated frontend service responsible for communication with the backend. Sends HTTP requests to `/translation-provider-mapping`
Responsibilities include:

- Fetching existing provider mappings.
- Submitting updated mappings when administrators modify language configurations.

This modular design ensures separation of concerns between UI components, authentication logic, and backend communication. It also follows the same architectural pattern used in other Oppia admin pages.

TranslateTextBackendApiService

The existing located at:

core/templates/pages/contributor-dashboard-page/services/[translate-text-backend-api.service.ts](#)

Will be extended.

Fetching the Translation

I will add a new method called `getMachineTranslationAsync()`. This method will send a POST request to the `/generate-translation` endpoint to fetch the AI-generated translation when the user clicks the “Auto-translate” button.

I will update the existing `suggestTranslatedTextAsync()` method to include two additional fields in the payload: `was_auto_generated` and `auto_generation_provider`. These fields will be added to the `change_cmd` so that reviewers can see whether the translation was generated by the AI system.

TranslationModalComponent

The

[oppia/core/templates/pages/contributor-dashboard-page/modal-templates/translation-modal.component.ts](#) will be enhanced to manage the state of the auto-translation workflow. The following changes will be made:

Will add state variables like `isAutoTranslationLoading`, `wasAutoGenerated`, and `autoGenerationProvider` to track auto-translation status. Implement a `generateAutoTranslation()` method triggered by the Auto-translate button, which calls the backend service, populates the editor, and sets metadata flags. Update `suggestTranslatedText()` to detect unedited AI text and show a confirmation modal before submission, and to pass `wasAutoGenerated`, `autoGenerationProvider`, and `wasEdited` to the backend. Extend `canTranslatedTextBeSubmitted()` to enforce alt-text review for images when the translation is auto-generated.

Documentation changes

The Oppia wiki will be updated with a new page titled:

Machine Translation Infrastructure

The documentation will include:

- Explanation of the translation provider registry
- Description of the HTML pre-processing mechanism used to protect Oppia components.
- Instructions for administrators to update the machine_translation_providers.json configuration file

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
auto_translation_button_clicked	target <i>languagecode</i> (string), <i>exploration_id</i> (string)	No. Needs to be added to core/domain/event_services.py and tracked in the frontend when the “Auto-translate” button is clicked.
auto_translation_suggestion_saved	was_edited (boolean), provider_name(string)	No. We need to extend the frontend save event payload to include whether the user modified the AI-generated text before saving, and the provider used.

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1

Key objective for this milestone: The backend infrastructure for automatic translation suggestions is fully implemented. The system can safely protect Oppia components, generate translations using the Azure API, and store them in the datastore cache. The Translator Admin page allows administrators to manage configurations.

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	Feature Flag and Static Config: Define ENABLE_AUTOMATIC_TRANSLATION_SUGGESTIONS in feature_flag_list.py to gate the feature during development.	None	May 26	Jun 2

	Create the static whitelist machine_translation_providers.json. machine_translation_provider_display_names.json			
2.	Datastore Model and Domain Objects : Define MachineTranslationPolicyModel in gae_models.py with MACHINE_TRANSLATION_POLICY_ID constant, MachineTranslationPolicyModel.get(..., strict=False) method, JsonProperty and BooleanProperty fields. Write service methods: is_automatic_translation_enabled(), Define MachineTranslationProviderMapping domain class in core/domain/translation_domain.py with validate(). get_machine_translation_provider_mapping(), save_machine_translation_provider_mapping(), update_automatic_translation_status Write unit tests for model, domain object, and all three service methods	1	Jun 1	Jun 9
3.	Abstract Base Class , API and Emulator: Implement the Azure Translator API adapter with exponential backoff and create the TranslateEmulator for CI/CD and local testing.	1,2	Jun 7	Jun 14
4.	Pre/Post-processing Engine and Tests : Implement HTML parsing , translate = "no" attribute injection, attribute stripping, and HTML sanitization integration.	3	Jun 11	Jun 17
5.	Registry, Caching and Domain Tests: Implement TranslationProviderRegistry and MachineTranslationGeneratorHandler. Integrate AutoTranslationCacheModel for caching. Write unit tests ensuring correct provider selection, whitelist validation, and proper handling of unsupported languages.	1,2,3,4	Jun 14	June 20
6.	Provider Mapping Handler (Backend): Implement the GET and PUT logic for /translation-provider-mapping to securely update the active Datastore mapping. Write backend controller tests.	2	Jun 16	Jun 22
7.	Admin Configuration UI and Acceptance Tests: Build the Translator Admin page UI to toggle the feature and map providers (implementing immediate save on row addition/deletion). Write	6	Jun 17	Jun 24

	acceptance tests verifying the admin can enable the feature and map providers successfully.			
–	<i>Note: At this point, the backend is fully functional. Translations can be generated, cached, and protected correctly via direct API requests. The Admin page is fully interactive.</i>			
	(Example of “demo” row) <u>Demo to PMs/stakeholders:</u> Demo the Translator Admin configuration page and showcase backend translation generation using a script or network tool.	1,2,3,4,5,6,7	Jun 26	

Milestone 2

Key objective for this milestone: Contributors can generate AI-based translation suggestions directly from the Contributor Dashboard. The interface allows users to review, edit, or reject translations, with mandatory accessibility safeguards for image alt-text

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	Frontend API Integration: Extend TranslateTextBackendApiService. Update the suggestion payload to accept was_auto_generated and auto_generation_provider. Include frontend unit tests.	M1 Completion	Jul 8	Jul 15
2	Contributor Dashboard UI (Part 1) and UI Tests: Add the "Auto-translate" button, populate the editor, and display badges. Write acceptance tests verifying the generation flow, ensuring math components and hyperlink URLs are preserved in the UI.	1	Jul 14	Jul 20
3	Contributor Dashboard UI (Part 2): Implement the confirmation popup for untouched AI text. Extend acceptance tests to cover the unedited confirmation modal flow.	2	Jul 19	Jul 25
4	Accessibility Safeguards and Final UI Tests: Implement the alt-text review block and disable the Save button appropriately. Extend acceptance tests to cover the enforced alt-text review journey.	3	Jul 25	Aug 5
	<u>At this point, the entire user journey is complete.</u>			

	<u>All flows (edited, unedited, alt-text, manual) are fully covered by acceptance tests written alongside their respective features..</u>			
	Demo to PMs/stakeholders: End-to-end demo of a contributor generating an AI translation, encountering the alt-text safeguard, handling the unedited confirmation modal, and submitting the translation.	1,2,3,4	Aug 6	N/A

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Need sync between Android / Web?	Key stakeholders
1.	ENABLE_AUTOMATIC_TRANSLATION_SUGGESTIONS	Enables the auto-translation feature in the Contributor Dashboard. The backend handler checks this flag at runtime and rejects requests if it is not enabled.	No (Web only)	Translation admins, Oppia maintainers, PM team

Future Work

Allow only selected people to allow this feature of auto translation

We should use the same permission system for translation suggestions as we use for questions. Users must be approved by translation admins before they can suggest translations. This means users cannot do anything in the Contributor Dashboard unless they have submitted or reviewer permissions from the admin dashboard. So, we will only show the Contributor Dashboard to users who have these permissions.

Automated Quality Scoring

A lightweight evaluation model could assign confidence scores to generated translations. Suggestions that fall below quality threshold can be highlighted for additional review and knowing the quality of automatic translations.

Extending the Translation Cache with Approved Human Translations

The translation caching system could be extended to store both machine-generated translations and approved human translations in the AutoTranslationCacheModel. When an approved translation exists for a given (source_text, language) pair, it could be prioritized over machine translations when generating suggestions. This idea aligns with the proposal discussed in Oppia issue <https://github.com/oppia/oppia/issues/19680> and would allow the suggestion system to improve translation quality over time.